

Reviewing Encrypted N2N (NCFED) Protocol Packets

Field guide to the certificate-secured federation channel on the wire

CAPTURE	captures/n2n-encrypted-20260717.pcap md5 3e3e73c2709f7e34b78be45178a5efb5
WINDOW	2026-07-17, 13:28:44–13:33:17 EDT (273 s · 292 frames · Linux cooked v2)
LOCAL NODE	as65007-7.7.7.7 (192.168.1.227)
REMOTE PEER	as65099-10.255.255.1 "Byrn", domain-verified netclaw.byrnbaker.me, via ngrok tunnel 8.tcp.us-cal-1.ngrok.io:xxxxx
CONTEXT	First capture of the NCFED federation channel after spec 060 (claw-cert security) enabled TLS. Companion to the 2026-07-14 cleartext capture (n2n-session-20260714.pcap), used throughout as a before/after contrast.

1 What you are looking at

An NCFED connection since spec 060 has **three wire phases** on a single TCP stream:

TCP 3-way handshake

Phase 1 Cleartext NCFED preamble — 13 bytes each direction

Phase 2 TLS 1.3 handshake — ALPN ncfed/1

Phase 3 TLS application data — JSON-RPC 2.0, fully encrypted

The preamble is deliberately kept cleartext so the daemon's single listening port can discriminate NCFED from BGP/NCTUN traffic before TLS starts (bgp/federation/tls.py:169 ; magic defined at bgp/constants.py:195 , preamble built at bgp/federation/channel.py:291–293).

2 Phase 1 — the cleartext NCFED preamble

13 bytes, sent by **both** sides immediately after TCP connect:

Offset	Size	Field	Value in this capture (initiator, frame 99)
0	5	Magic "NCFED"	4e 43 46 45 44
5	4	Local AS (!I , big-endian u32)	00 00 fd ef = 65007
9	4	Router-ID (packed IPv4)	07 07 07 07 = 7.7.7.7

Responder (frame 102): 4e434645 44 | 0000fe4b | 0affff01 → AS 65099, router-ID 10.255.255.1.

REVIEWER TAKEAWAY

Peer *identities* (AS + router-ID) are visible on the wire even after 060. This is by design (port discrimination), but it means a passive observer can enumerate who federates with whom.

Find it: `frame` contains `"NCFED"` — in this capture, frames 99 (out) and 102 (in).

3 Phase 2 — TLS 1.3 handshake

Observed in frames 104–113 (connection opened at t=92.5 s by a forced redial):

Property	Value	Where to see it
TLS version	1.3 (supported_versions: 0x0304)	ClientHello/ServerHello ext
ALPN	ncfed/1	ClientHello ext (cleartext)
SNI	netclaw.byrn baker.me	ClientHello ext (cleartext)
Cipher chosen	TLS_AES_256_GCM_SHA384 (0x1302)	ServerHello
Key share offered	X25519MLKEM768 (0x11ec, PQ hybrid) + X25519	ClientHello key_share
Key share chosen	X25519 (29) — server declined the PQ hybrid	ServerHello key_share
Certificates	Not visible — TLS 1.3 encrypts them after ServerHello	(absent from <code>tls.handshake.type==11</code> filter)

REVIEWER TAKEAWAYS

- The SNI carries the peer's **claw domain** — the domain-verified identity from spec 060 is observable in cleartext (standard TLS behavior; ECH is not in use).
- ALPN `ncfed/1` is a reliable protocol fingerprint for NCFED traffic on any port.
- Our side offers hybrid post-quantum key exchange; whether it's used depends on the peer. In this capture the peer (Byrn) negotiated classical X25519.
- Because TLS 1.3 encrypts the Certificate message, **you cannot audit the peer certificate from a passive pcap**. Use the daemon's `/n2n/certs` endpoint or an inline inspection point instead.

4 Phase 3 — encrypted application data

Everything after the handshake is TLS records of opaque type 23 (Application Data). In this capture the fresh connection carried **69 app-data records, ~109 KB total** (~57.8 KB from Byrn, ~51.5 KB from us) covering: inventory announcement, a chat open/reject exchange, a delegated-task submit, and ~15 s-interval task-status polls.

What is still inferable from ciphertext (traffic analysis):

- **Record sizes.** The inventory transfer shows as a burst of 2 968-byte records; JSON-RPC polls show as ~170–300-byte records. Message *type* can often be guessed from size/cadence.
- **Timing.** Poll intervals, task duration, and interactive vs. idle phases are visible.

- **Nothing else.** `strings` over the payloads yields only `NCFED` (preamble) and `ncfed/1` (ALPN) — no JSON-RPC keys, methods, identities, or results. Compare the 2026-07-14 cleartext capture, where frame payloads decoded directly to e.g. `{"jsonrpc":"2.0","id":"as65007-7.7.7.7:23","method":"n2n/end-point_update",...}` (see `analysis-20260714/payloads.txt`).

5 Adjacent traffic you will see in a real capture

- **Cleartext BGP mesh keepalives on loopback.** The BGP *mesh session* (peer discovery/routing layer) is separate from the NCFED channel and is **not** covered by 060. Inbound mesh traffic arrives via the local ngrok agent (`ngrok tcp 20203`) and shows on loopback as classic BGP: 19-byte messages, `16× 0xff` marker + length `0x0013` + type `0x04` (KEEPALIVE). Filter: `tcp.port==20203 && tcp.len==19`. In this capture: 10 keepalives from the session with `as65001` (John).
- **Superseded channels.** Long-lived NCFED connections whose handshake predates the capture show only mid-stream type-23 records (our old channel on port 44786 here). `tshark` still tags them `tls` but you will not see the handshake; identify the peer by IP/port.
- **ngrok indirection.** The remote IPs are ngrok edge nodes (e.g. `54.219.47.216`), not the peer's real address, and they change per tunnel restart. Filter by the tunnel **port**, not host. A stale tunnel address fails with TCP RST (connection refused) — if you see `SYN→RST` to an old port, the peer's tunnel moved.

6 Capture and analysis recipes

Capturing

```
sudo -b timeout 300 tcpdump -i any -s 0 -U \
-w /tmp/n2n-encrypted-$(date +%Y%m%d).pcap \
'tcp port <peer-tunnel-port> [or tcp port <other-peer-port>]'
```

Notes:

- Ports are per-peer ngrok tunnel ports — get the current ones from `curl -s 127.0.0.1:8179/n2n/health` (`endpoint` field).
- To guarantee a TLS handshake in the window, force a redial while capturing: `curl -s -X POST 127.0.0.1:8179/n2n/connect -d '{"peer":"...", "host":"...", "port":...}'`.
- **Ubuntu AppArmor gotcha:** the `tshark` profile only permits reading pcaps you *own* under `/tmp` (`abstractions/user-tmp`) — it cannot read `$HOME` paths or root/tcpdump-owned files (denials appear in `journalctl -k` as `apparmor="DENIED" profile="tshark"`). Copy the pcap to a file you own in `/tmp` before analysis.

Display filters (Wireshark or `tshark -Y`)

Goal	Filter
NCFED preambles	<code>frame contains "NCFED"</code>
TLS handshakes only	<code>tls.handshake</code>
ClientHello (SNI/ALPN/PQ offer)	<code>tls.handshake.type==1</code>
ServerHello (negotiated params)	<code>tls.handshake.type==2</code>
Leaked certificates (should be empty on TLS 1.3)	<code>tls.handshake.type==11</code>
Encrypted app data	<code>tls.record.opaque_type==23</code>
BGP mesh keepalives	<code>tcp.port==<mesh-port> && tcp.len==19</code>
New connections (redials)	<code>tcp.flags.syn==1 && tcp.flags.ack==0</code>

Field extractions used for this report

```
# Conversations overview
tshark -r x.pcap -q -z conv,tcp

# Handshake parameters
tshark -r x.pcap -Y "tls.handshake.type==1" -T fields \
  -e tls.handshake.extensions_server_name -e tls.handshake.extensions_alpn_str \
  -e tls.handshake.extensions_key_share_group -e tls.handshake.ciphersuite
tshark -r x.pcap -Y "tls.handshake.type==2" -T fields \
  -e tls.handshake.ciphersuite -e tls.handshake.extensions_key_share_group

# Plaintext-leak sweep (expect ONLY 'NCFED' and 'ncfed/1')
tshark -r x.pcap -Y 'tcp.payload' -T fields -e tcp.payload \
  | xxd -r -p | strings -n 6 | grep -iE 'jsonrpc|method|n2n/|inventory'

# App-data volume per direction
tshark -r x.pcap -Y "tls.record.opaque_type==23" -T fields -e ip.src -e tls.record.length \
  | awk '{split($2,a,","); for(i in a) b[$1]+=a[i]} END {for(k in b) print k, b[k]}'
```

7 Review checklist

For each NCFED connection in a capture, verify:

- 1 Preamble sanity** — exactly 13 bytes each way; AS/router-ID match the expected peers; nothing else precedes TLS.
- 2 TLS 1.3 negotiated** (`supported_versions 0x0304` in ServerHello) — a 1.2 downgrade or missing TLS entirely is a red flag.
- 3 ALPN is `ncfed/1`** — its absence suggests a non-NCFED client or downgrade attempt.

- 4 **SNI matches the peer's registered claw domain** (cross-check `/n2n/peers/<id>/trust` → `claw_domain`).
- 5 **No Certificate message in cleartext** (`tls.handshake.type==11` empty).
- 6 **No plaintext JSON-RPC** — the strings sweep must return only `NCFED` / `ncfed/1`.
- 7 **AEAD cipher** — expect `TLS_AES_256_GCM_SHA384` or another TLS 1.3 suite; anything CBC/legacy is a failure.
- 8 **Note the key-exchange group** — record whether the PQ hybrid (0x11ec X25519MLKEM768) was accepted; classical X25519 means the peer's stack lacks PQ support.

8 Observations from this capture (2026-07-17)

- PASS** Full encryption of the NCFED channel confirmed; zero JSON-RPC leakage (vs. total leakage in the 07-14 capture).
- PASS** TLS 1.3 + AES-256-GCM; certificates not exposed.
- WARN** Peer declined the post-quantum hybrid key share — Byrn's stack negotiated classical X25519 only.
- WARN** Peer AS + router-ID (preamble) and claw domain (SNI) remain observable — accepted metadata exposure, but reviewers should know it exists.
- WARN** The BGP **mesh session** keepalives are still cleartext BGP (out of 060's scope; rides inside ngrok's own transport on the WAN leg). Candidate for a future spec if mesh-layer confidentiality matters.
- INFO** Stale ngrok endpoints present as connection-refused (`4.tcp.us-cal-1.ngrok.io:20941` in this session); the daemon recovers only after a manual `/n2n/connect` to the peer's newly advertised endpoint — manual dials do not persist `endpoint_host` / `endpoint_port` to the peer record.

Produced from a live two-node federation session (as65007 ↔ as65099). Traffic generated during capture: forced redial (TLS handshake), chat open/send (rejected — chat disabled peer-side), delegated `cml-admin` task submit + status polls. Additional concurrent checks were run by peer-side agents (OpenClaw Codex, Hermes).